

Entrega 1

Lideranças Empáticas (LE) **Contagem Inteligente de Alimentos com Visão Computacional**

Curso	Ciência da Computação - 5º semestre
Equipe	Antônio Petri de Moraes Soares de Moura e Oliveira; Leonardo Santos da Silva; Lucas de Lima Gutierrez; Vitor Kenzo Kanashiro
Tema	Solução integrada para classificação e contagem automática de alimentos arrecadados

Março de 2026

Resumo Executivo

Este documento apresenta a concepção completa do produto Lideranças Empáticas - Contagem Inteligente de Alimentos, com foco no problema operacional, escopo do MVP, arquitetura técnica, critérios de validação e planejamento inicial. Além disso, a entrega detalha o frontend já modelado no repositório do projeto, descrevendo sua estrutura Flutter, rotas, gerenciamento de estado, integração com APIs, telas por perfil e fluxo operacional. O objetivo é permitir que qualquer leitor compreenda a solução proposta, a divisão de responsabilidades do sistema e o estágio atual de implementação.

Definição do problema operacional

No contexto do projeto Lideranças Empáticas, a contagem de alimentos arrecadados pelas equipes é, em sua forma tradicional, um processo manual. Essa abordagem depende diretamente da atenção humana, o que aumenta a probabilidade de erros como contagens duplicadas, omissões, registros inconsistentes e dificuldade de auditoria posterior.

Além do risco operacional, a contagem manual consome tempo, desacelera a triagem e dificulta a consolidação dos dados quando múltiplas equipes participam simultaneamente da arrecadação. Isso reduz a confiabilidade da informação e limita a geração de relatórios comparativos entre equipes, categorias e períodos.

Diante desse cenário, o projeto propõe uma solução digital integrada que automatiza o reconhecimento e a contagem dos alimentos, centraliza os registros e oferece apoio visual e gerencial para operação, coordenação e administração.

Objetivo geral

Desenvolver uma solução integrada composta por câmera, Visão Computacional/IA, backend e aplicativo mobile, capaz de identificar, classificar e contabilizar pacotes de alimentos arrecadados, registrando automaticamente a produção por equipe, categoria, data, hora e nível de confiança da predição.

Objetivos específicos

- Permitir autenticação de usuários e organização por perfis operacionais.
- Selecionar a equipe arrecadadora ativa antes do início da sessão.
- Realizar captura contínua de imagens em ambiente controlado.
- Classificar automaticamente os itens em arroz, feijão e outros no MVP.
- Evitar duplicidade de contagem por meio de estabilidade e janela de resfriamento.
- Persistir leituras com equipe, categoria, operador, confiança e timestamp.
- Oferecer visualização, metas, filtros, exportação e administração por perfil.

Escopo do MVP

O Produto Mínimo Viável concentra-se em provar que a abordagem técnica funciona de ponta a ponta. No MVP, o sistema deve identificar visualmente três classes principais - arroz, feijão e outros - em um cenário controlado, registrando as leituras e relacionando-as à equipe selecionada.

O escopo funcional inclui autenticação, definição de perfis, seleção de equipe, captura por câmera, classificação do item, contagem automática, armazenamento das leituras, acompanhamento de metas, visualização de registros e exportação de dados filtrados em CSV.

Elementos mais avançados, como dashboard web completo, múltiplas categorias adicionais, inferência offline no próprio dispositivo, rastreamento avançado por objeto e deploy final com observabilidade plena, ficam previstos como evolução após a validação do MVP. **Público usuário**

Perfil	Responsabilidade principal	Funcionalidades esperadas
Operador	Executar a contagem em campo	Selecionar equipe, usar câmera, visualizar leituras e acompanhar metas.
Coordenador	Acompanhar a operação	Consultar tabela de dados, exportar relatórios e administrar equipes.
Administrador	Governança do sistema	Criar usuários, administrar equipes, configurar metas, consultar dados e exportar relatórios.

3. Jornada de uso

A jornada do usuário começa com a autenticação no aplicativo. Após o login, o sistema identifica o perfil do usuário e o direciona para a home correspondente ao seu papel no processo.

No caso do operador, a sequência operacional segue a lógica: entrar no sistema, selecionar a equipe ativa, abrir a câmera, capturar automaticamente os alimentos, validar a estabilidade das predições e registrar as leituras confirmadas. Ao final, essas leituras ficam disponíveis para consulta, comparação com metas e exportação pelos perfis autorizados.

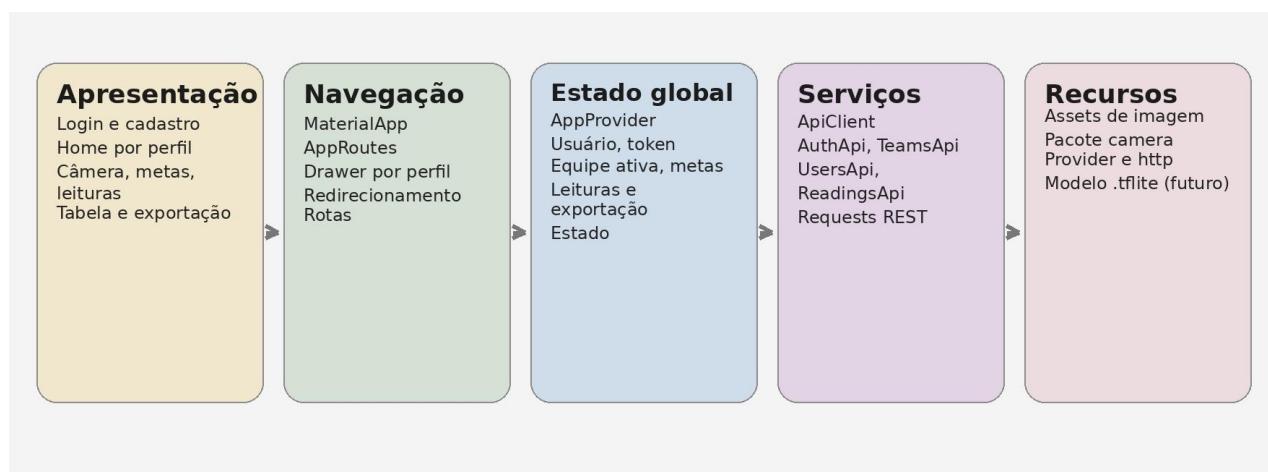


Figura 1 - Jornada resumida do usuário dentro do sistema.

Requisitos funcionais e não funcionais

Requisitos funcionais

- Cadastrar e autenticar usuários com perfis distintos.
- Selecionar equipe ativa antes da sessão de contagem.
- Capturar imagens continuamente por câmera.
- Executar classificação automática dos itens.
- Confirmar leituras com controle de duplicidade.
- Registrar eventos de leitura no backend.
- Consultar histórico, metas e dados filtrados.
- Exportar leituras em CSV.
- Gerenciar equipes, metas e usuários conforme o perfil.

Requisitos não funcionais

- Baixa latência para feedback próximo do tempo real.
- Arquitetura modular para facilitar manutenção e evolução.
- Comunicação via API REST autenticada por token.
- Escalabilidade para inclusão de novas classes e relatórios.
- Usabilidade simples para operação em campo.
- Rastreabilidade por meio de timestamp, equipe e operador.

Arquitetura geral da solução

A arquitetura proposta é dividida em quatro blocos principais: captura, processamento inteligente, persistência e apresentação. A câmera faz a aquisição do frame; o módulo de IA/VC classifica o alimento e valida a leitura; o backend registra o evento; e o aplicativo disponibiliza os resultados ao usuário.

Essa separação facilita a evolução incremental do sistema, permitindo trocar a estratégia de inferência, ampliar o modelo de dados e melhorar a experiência do operador sem reescrever toda a solução.

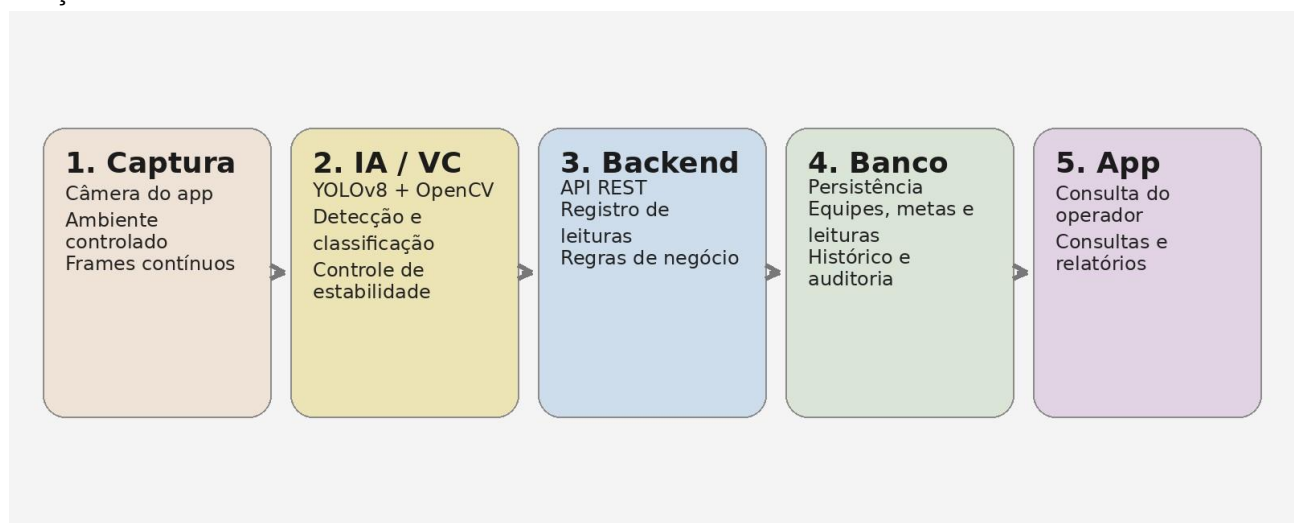


Figura 2 - Arquitetura geral da solução proposta.

Frontend do projeto

O frontend do projeto está organizado em Flutter dentro da pasta SRC/App. Ele representa a camada de interação entre usuários e o restante da solução, e já contempla autenticação, navegação por perfil, captura por câmera, visualização de dados, gerenciamento de equipes, metas, usuários e exportação de registros.



Figura 3 - Organização lógica do frontend Flutter.

Stack e dependências do app

Tecnologia	Uso no app	Observação
Flutter/Dart	Base do aplicativo mobile/web	Responsável por UI, navegação e integração.
provider	Gerenciamento de estado	Centraliza usuário, token, equipe, leituras e metas.
http	Consumo de API REST	Usado pelo ApiClient e serviços especializados.
camera	Captura de imagem	Usado na tela de leitura automática.
image_picker / image / mime / http_parser	Apoio a arquivos e upload	Preparação para cenários com captura e envio de imagem.
tflite_flutter	Suporte futuro a inferência local	Já está declarado, embora o fluxo atual use chamada de API para leitura.

Ponto de entrada e inicialização

O arquivo main.dart inicializa o aplicativo com MultiProvider e registra o AppProvider como estado global. Em seguida, o MaterialApp é criado com o título Lideranças Empáticas, tema centralizado e rota inicial apontando para a tela de login.

Esse desenho garante que, desde o primeiro frame, informações de autenticação, perfil, equipe ativa, metas e leituras possam ser compartilhadas entre as telas sem duplicação de lógica.

Tema visual e identidade

O aplicativo utiliza uma identidade simples e coerente, baseada em um tema claro com cor primária laranja (#FF6B00), cor secundária amarela e destaque verde para mensagens de sucesso. A imagem foods_bg.png é usada como fundo nas telas de autenticação, reforçando o contexto do projeto. Essa escolha visual ajuda a manter consistência entre login, cadastro e áreas operacionais.

Navegação e rotas

A navegação é centralizada em core/routes/app_routes.dart. As rotas foram organizadas por domínio e por perfil, permitindo separar claramente telas públicas, telas do operador e telas de coordenação/administração.

Grupo	Rotas principais	Finalidade
Autenticação	/login, /register, /profile/edit	Entrada do usuário, cadastro e edição de perfil.
Homes por cargo	/home/operador, /home/coordenador, /home/admin	Redirecionamento conforme papel do usuário.
Operação comum	/camera, /team, /readings, /goals	Leitura, seleção de equipe, histórico e acompanhamento.
Coordenação/Admin	/manage-teams, /data-table, /export	Gestão de equipes, consulta tabular e exportação.
Administração	/admin/manage-goals, /admin/manage-users	Configuração de metas e governança de contas.

Gerenciamento de estado com AppProvider

O AppProvider é a peça central do frontend. Ele mantém o papel do usuário (operador, coordenador ou admin), nome, email, token de acesso, lista de equipes, equipe ativa, leituras registradas e metas configuradas.

Também concentra regras úteis para a interface, como cálculo da rota inicial por papel, atualização local do perfil, inserção de leituras, contagem por equipe/categoria, cadastro e remoção de metas, geração de CSV e processo de logout.

Essa abordagem reduz acoplamento entre telas e permite que a interface reaja automaticamente quando algum dado muda, por exemplo, quando a equipe ativa é alterada ou quando uma nova leitura é registrada.

Camada de integração com API

O frontend possui uma camada de serviços em `core/api`. O `ApiClient` encapsula requisições GET, POST, PUT e DELETE, injeta o cabeçalho Authorization com Bearer token quando disponível e utiliza uma URL base centralizada.

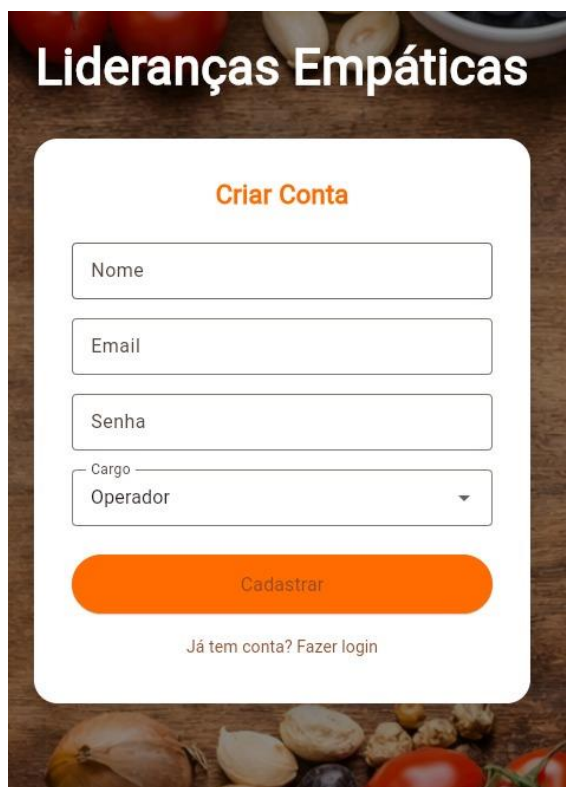
Serviço	Endpoints associados	Função
AuthApi	/api/auth/register, /api/auth/login, /api/auth/me	Cadastro, login e leitura do perfil autenticado.
TeamsApi	/api/teams	Listar, criar e remover equipes.
UsersApi	/api/users, /api/users/{id}	Listar, criar, atualizar e excluir usuários.
ReadingsApi	/api/readings	Registrar leitura confirmada no backend.

Atualmente, a URL base da API está definida diretamente em `api_config.dart`, apontando para um endpoint HTTP do backend. Para produção, recomenda-se mover essa configuração para ambientes e variáveis dedicadas.

Fluxo de autenticação

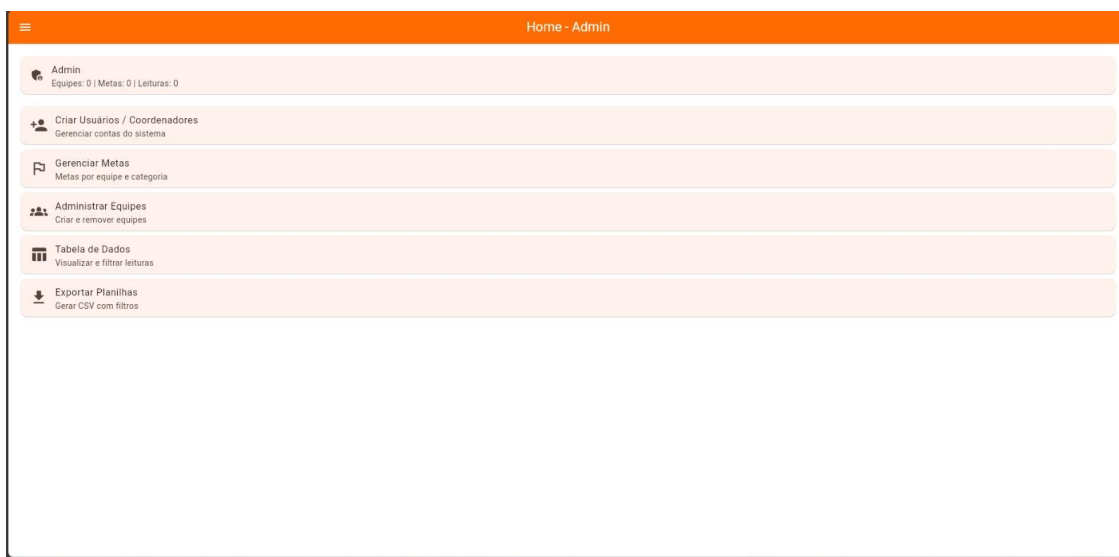
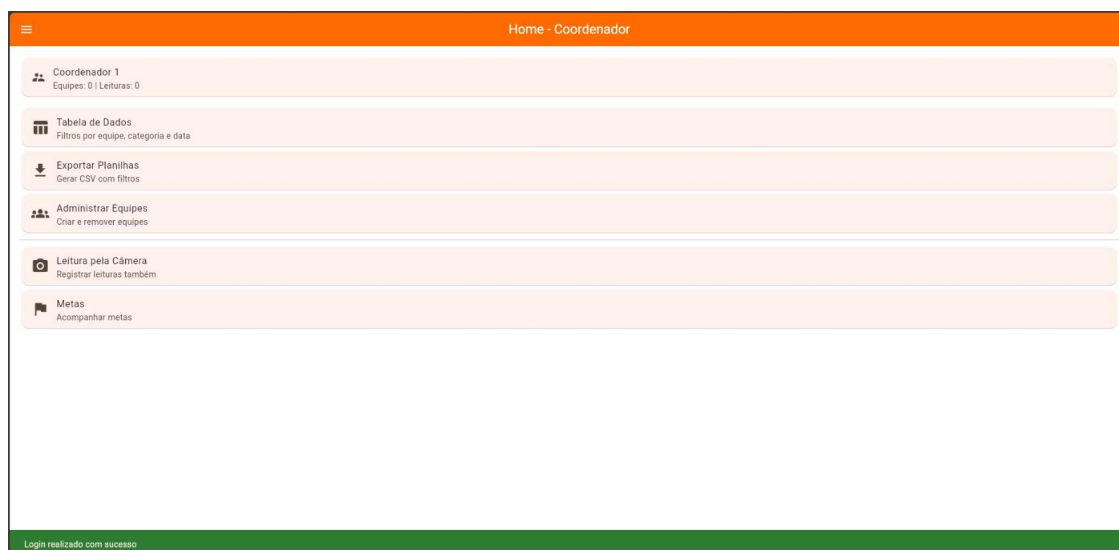
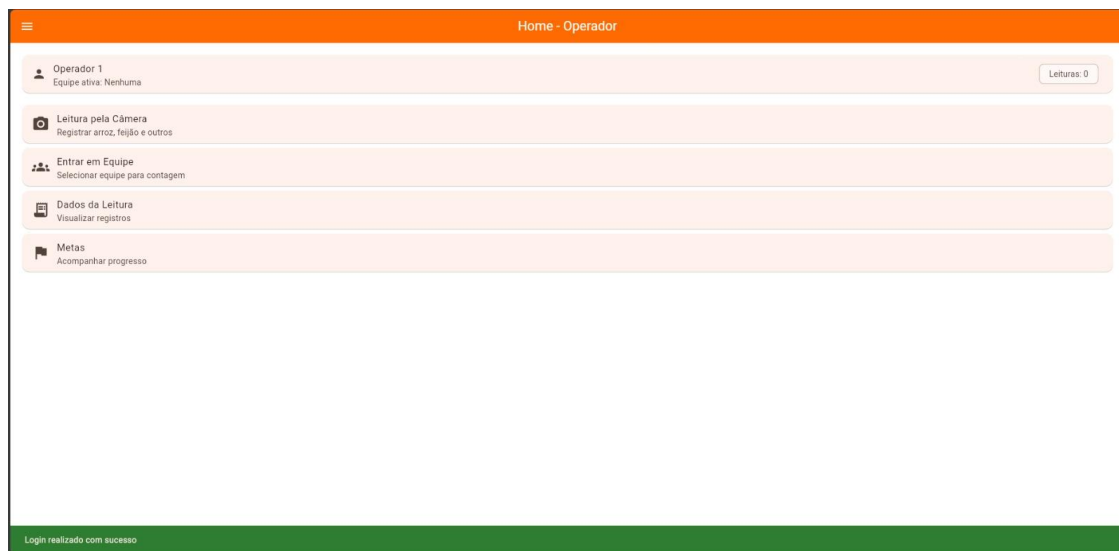
Na `LoginPage`, o usuário informa email e senha. O app chama `AuthApi.login`, recebe o token, salva-o no `AuthProvider` e em seguida consulta `/api/auth/me` para recuperar nome, email e papel real do usuário. Com isso, o aplicativo redireciona automaticamente para a home correta.

Na `RegisterPage`, o usuário preenche nome, email, senha e cargo. O cadastro é enviado ao backend e, após a confirmação, o sistema também consulta o perfil para definir corretamente o fluxo pós-cadastro.



Estrutura das telas por perfil

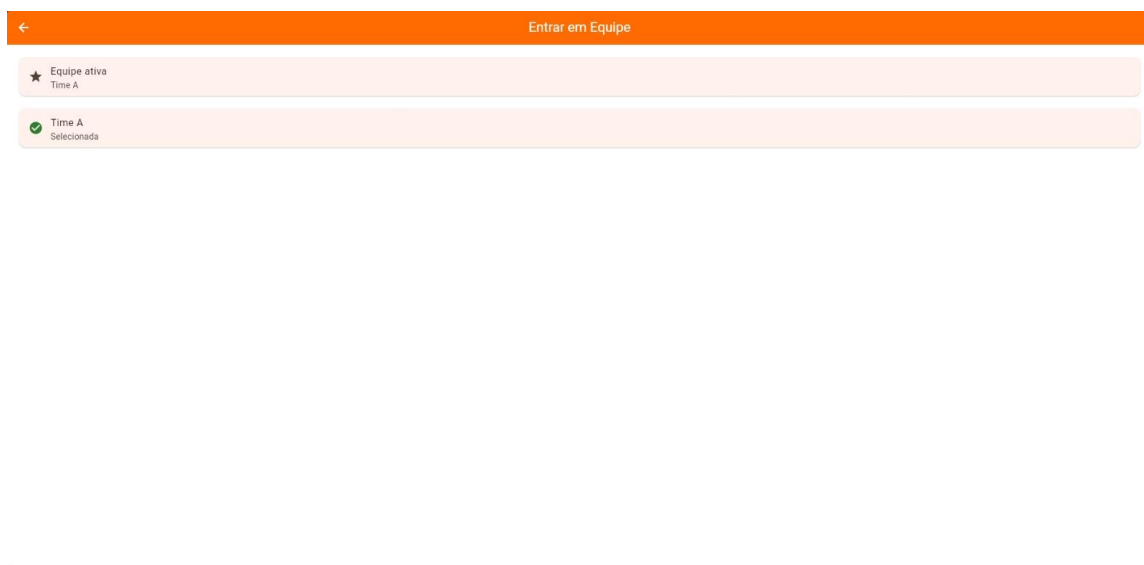
Tela	Perfil	Função principal	Comportamento
HomePage	Operador	Centro operacional	Exibe usuário, equipe ativa, total de leituras e atalhos para câmera, equipe, leituras e metas.
CoordinatorHomePage	Coordenador	Acompanhamento da operação	Atalhos para tabela, exportação, equipes, câmera e metas.
AdminHomePage	Admin	Governança do sistema	Acesso a usuários, metas, equipes, tabela e exportação.
AppDrawer	Todos	Navegação contextual	Mostra dados do usuário e libera menus conforme o papel.





Tela de equipe e contexto operacional

A TeamPage lista as equipes carregadas no AppProvider e permite definir qual será a equipe ativa da sessão. Esse passo é obrigatório para o fluxo ideal da leitura, pois a câmera utiliza essa informação para relacionar cada evento de contagem a uma equipe específica.



Tela de câmera e leitura automática

A CameraPage é o núcleo operacional do frontend. Ela inicializa a câmera traseira, cria um CameraController, ativa um temporizador periódico e realiza capturas automáticas em intervalos regulares.

Cada captura é enviada ao método predict do ReadingsApi. Em seguida, a interface exibe categoria prevista, confiança e contador de estabilidade. Somente quando a mesma classe se repete por uma quantidade mínima de frames e respeita um tempo de resfriamento a leitura é confirmada.

Depois da confirmação, a tela chama createReading e envia ao backend o team_id, a categoria, a confiança e o image_path. Esse desenho diminui o risco de contagens duplicadas ao aplicar uma lógica simples de estabilidade e cooldown.

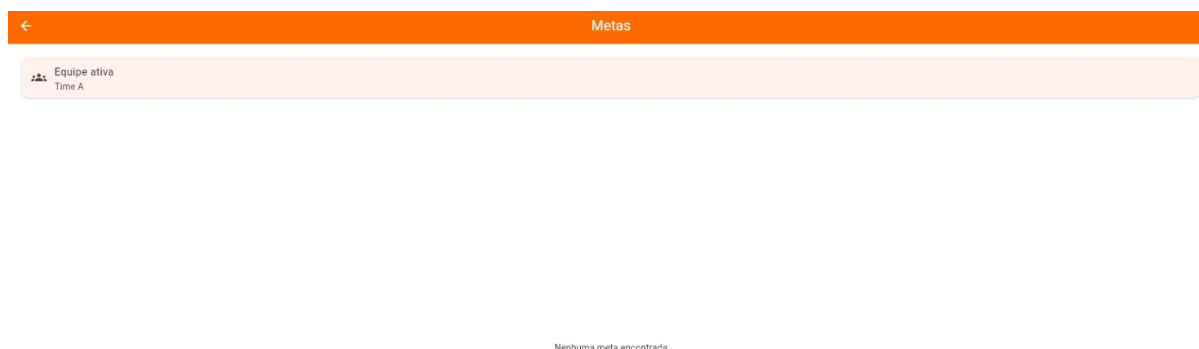
No estado atual do código, o método predict do frontend ainda está mockado, retornando uma resposta fixa de categoria outros com confiança 0,80. Isso significa que a interface de captura e o fluxo de registro já estão estruturados, mas a integração real com o motor de inferência ainda deve ser concluída.

Leituras, metas, tabela e exportação

A ReadingsPage mostra a lista de eventos registrados, ordenados de forma a facilitar a consulta do histórico. A GoalsPage cruza metas cadastradas com a contagem atual por equipe e categoria, apresentando progresso com barras percentuais.

A DataTablePage amplia a capacidade analítica do frontend. Ela permite filtrar leituras por equipe, categoria e faixa de datas, exibindo os resultados em uma PaginatedDataTable. Esse componente já traduz o projeto em uma ferramenta de acompanhamento gerencial.

A ExportPage reutiliza os filtros do AppProvider e gera um CSV com as leituras filtradas. No navegador, o download é direto; em dispositivos Android, o fluxo foi pensado para salvar o arquivo localmente.



←

Tabela de Dados

Equipe

Todas

Categoria

Todas

📅

Data inicial: --

📅

Data final: --

Registros: 0

Limpar filtros

Leituras

Equipe	Operador	Categoria	Data/Hora	Confiança

1-0 of 0 < >

←

Exportar Planilhas

Equipe

Todas

Categoria

Todas

📅

Data inicial: --

📅

Data final: --

Limpar filtros

📄

Gerar CSV

No Android o arquivo é salvo nos Documentos do app.

No Web o download é direto pelo navegador.

Gestão administrativa

A `ManageTeamsPage` consome a API de equipes para criar, listar e remover times. A `ManageGoalsPage` permite configurar metas por equipe e categoria diretamente sobre o estado global do aplicativo, com cálculo do progresso atual.

A `ManageUsersPage` oferece ao administrador uma interface completa para governança de contas: criação de usuários, definição de papel, ativação/inativação e exclusão. Esse módulo se integra ao `UsersApi` e amplia o caráter real de sistema corporativo do MVP.

A `EditProfilePage` permite atualização local de nome e email, reforçando a experiência do usuário dentro do app.

←

Administrar Equipes

Nome da equipe

Criar equipe

👤

Time A

←

Gerenciar Usuários

Criar novo usuário

Nome

Email

Senha

Perfil
Operador

Usuário ativo

Criar usuário

Admin
admin@admin.com

Ativo

Operador 1
op1@email.com

Ativo

Coordenador 1
coord1@email.com

Ativo

Editar Perfil

Nome

Operador 1

Email

op1@email.com

Salvar

Limitações atuais do frontend e próximos passos

- Conectar o método predict a um endpoint real de inferência ou motor local de IA.
- Persistir token e sessão de modo completo em armazenamento seguro.
- Sincronizar metas também com backend, em vez de apenas estado local.
- Adicionar evidência visual da leitura confirmada.
- Aprimorar tratamento de falhas de rede, permissões da câmera e estados offline.

Backend, dados e arquitetura de integração

A proposta completa do projeto considera um backend responsável por autenticação, cadastro de equipes, gerenciamento de usuários, recebimento das leituras e disponibilização de dados para consulta e exportação. O frontend analisado já está preparado para consumir uma API REST com esses domínios.

No modelo operacional desejado, cada leitura deve conter ao menos: identificador da equipe, categoria classificada, timestamp, operador responsável, confiança da predição e referência para eventual evidência. Essa estrutura garante rastreabilidade e abre espaço para relatórios gerenciais e auditoria.

Modelo conceitual de dados

Entidade	Campos essenciais	Finalidade
Usuário	id, nome, email, role, active	Controle de acesso e responsabilidades.
Equipe	id, name	Organizar arrecadação e comparar desempenho.
Leitura	team_id, category, confidence, timestamp, operator	Histórico operacional do sistema.
Meta	team_id, category, target	Acompanhamento de desempenho por categoria.

Classes reconhecidas e estratégia de contagem sem duplicidade

Para a Entrega 1, as classes definidas no MVP são arroz, feijão e outros. Essa escolha equilibra relevância operacional e viabilidade técnica para coleta de dados, treinamento e validação.

A estratégia atual de contagem sem duplicidade combina duas ideias simples e eficazes: estabilidade e cooldown. Primeiro, a mesma predição precisa repetir-se por vários frames consecutivos acima de um limiar mínimo de confiança. Depois, uma janela de resfriamento impede que a mesma classe seja confirmada repetidamente em poucos segundos.

Como evolução, o projeto pode incorporar rastreamento por objeto, linha virtual de passagem e validação geométrica da região de interesse, aumentando robustez quando houver fluxo contínuo ou esteira.

Critérios iniciais de validação e métricas

A avaliação técnica da solução deve considerar tanto métricas de classificação quanto indicadores operacionais. Para o modelo de IA, recomenda-se utilizar acurácia, precisão, recall, F1-score e matriz de confusão por classe.

No nível do sistema, também devem ser observados: taxa de contagem correta por sessão, número de duplicidades evitadas, latência média entre captura e registro, estabilidade da câmera e consistência dos dados persistidos no backend.

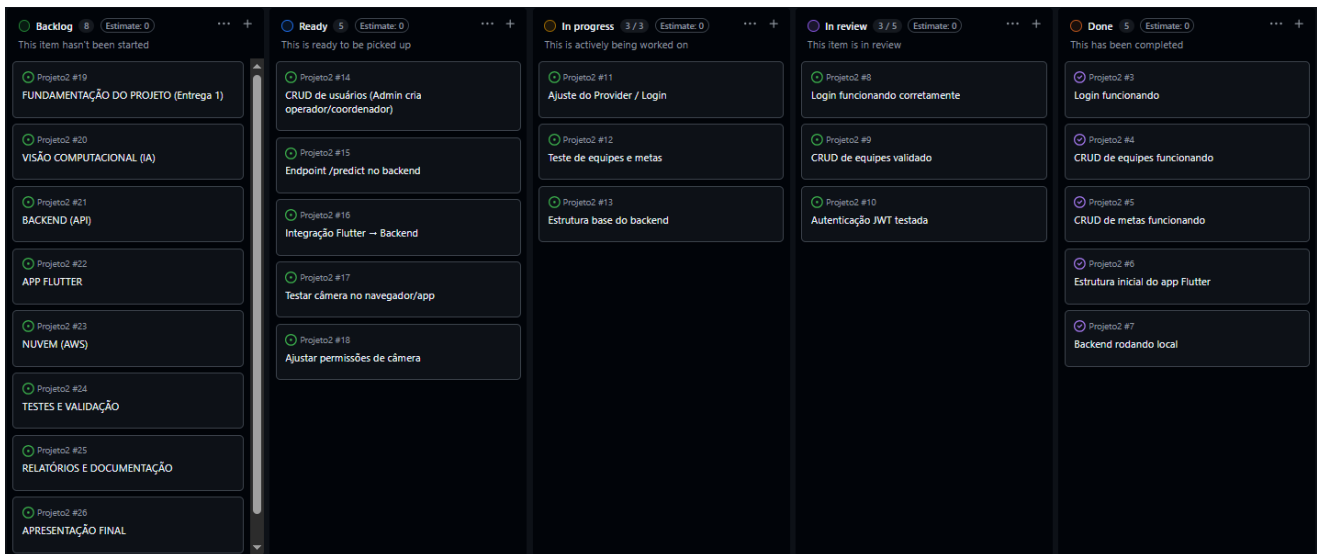
Planejamento técnico inicial e roadmap

Fase	Objetivo	Entregas esperadas	Risco principal
Fase 1	Definir arquitetura e requisitos	Documento conceitual, fluxos e papéis	Escopo excessivo.
Fase 2	Coletar e rotular dados	Dataset inicial e baseline de classes	Pouca variedade de amostras.
Fase 3	Validar PoC de IA	Modelo inicial e métricas preliminares	Baixa precisão ou instabilidade.
Fase 4	Integrar backend e app	Leitura persistida por equipe	Falhas de contrato entre API e app
Fase 5	Aprimorar UX e relatórios	Metas, filtros, exportação e evidências	Complexidade crescente de manutenção.

Divisão inicial de papéis no time

- IA e Visão Computacional: coleta, rotulagem, treinamento, validação e pipeline de inferência.
- Frontend Flutter: autenticação, navegação, câmera, consultas, relatórios e exportação.

- Backend/API: usuários, equipes, leituras, persistência, segurança e integração.
- Documentação e integração: diagramas, validação de requisitos, testes e consolidação das entregas.



Análise crítica do estágio atual

O projeto já apresenta uma base frontend madura para um MVP acadêmico: existe separação por camadas, navegação organizada, gerenciamento global de estado, diferenciação de perfis, consumo de APIs e telas que dialogam diretamente com os requisitos da disciplina.

O principal ponto pendente está na conexão final entre captura e inferência real. Embora a tela da câmera possua lógica de estabilidade, confirmação e persistência, o retorno de classificação ainda é simulado no frontend. Portanto, a próxima etapa técnica é consolidar o serviço real de IA para transformar a prova de conceito em fluxo operacional completo.

Mesmo assim, a estrutura atual já demonstra visão sistêmica do produto, pois o frontend foi modelado não apenas como demo visual, mas como interface de operação, coordenação e gestão.

Considerações finais

A proposta do Lideranças Empáticas atende ao objetivo da Entrega 1 ao apresentar uma concepção completa do produto, articulando problema, usuários, fluxo de uso, arquitetura, critérios de validação e planejamento técnico. O detalhamento do frontend mostra que o projeto já possui uma base concreta de implementação alinhada ao problema real, com foco em usabilidade, rastreabilidade e escalabilidade. Com a integração definitiva do motor de IA e o amadurecimento do backend, a solução tem potencial para automatizar a contagem de alimentos com ganho real de precisão, eficiência e impacto social.